

Assumptions

- film_category primary key is a tuple of film_id, category_id
- film_actor primary key is a tuple of actor_id, film_id
- If some primary keys do not start at 1, that is because rows were deleted due to incomplete data
- payment is missing row with payment_id = 424
- rental is missing row with payment_id = 321
- unique (rental_date, inventory_id, customer_id) tuple for rental
- Stripped film of multiple values in last column

#1

```
1  #1 What is the average length of films in each category? List the results in alphabetic order of categories.
2  • select category.name as category_name, avg(length) as avg_length
3  from film
4  join film_category using (film_id)
5  join category using (category_id)
6  group by category_name
7  order by category_name;
8
```

0% 1:8 1 error found

Result Grid Filter Rows: Search Export:

category_name	avg_length
Action	111.6094
Animation	111.0152
Children	109.8000
Classics	111.6667
Comedy	115.8276
Documentary	108.7500
Drama	120.8387
Family	114.7826
Foreign	121.6986
Games	127.8361
Horror	112.4821
Music	113.6471
New	111.1270
Sci-Fi	108.1967
Sports	128.2027
Travel	113.3158

Uses the avg() aggregate function and the joined table of film, film_category, and category.

#2

```
9      #2 Which categories have the longest and shortest average film lengths?
10  with category_avg as (
11      select category.name as category_name, avg(length) as avg_length
12      from film
13      join film_category using (film_id)
14      join category using (category_id)
15      group by category_name
16  )
17  select *
18  from category_avg
19  where avg_length = (select max(avg_length) from category_avg)
20         or avg_length = (select min(avg_length) from category_avg)
21  order by category_name;
22
```

00% 1:22 1 error found

Result Grid Filter Rows: Search Export:

category_name	avg_length
Sci-Fi	108.1967
Sports	128.2027

Uses a CTE to compute the average per category, then uses max and min in a where statement to get the results down to the longest and shortest averages.

#3

```
23 #3 Which customers have rented action but not comedy or classic movies?
24 select first_name, last_name
25 from customer
26 join rental using (customer_id)
27 join inventory using (inventory_id)
28 join film using (film_id)
29 join film_category using (film_id)
30 join category using (category_id)
31 where category.name = 'Action'
32 except
33 select first_name, last_name
34 from customer
35 join rental using (customer_id)
36 join inventory using (inventory_id)
37 join film using (film_id)
38 join film_category using (film_id)
39 join category using (category_id)
40 where category.name = 'Comedy' or category.name = 'Classics';
41
```

100% 62:40 1 error found

Result Grid Filter Rows: Search Export:

first_name	last_name
LAWRENCE	LAWTON
MATTHEW	MAHAN
TOM	MILNER
JO	FOWLER
SCOTT	SHELLEY
EDWIN	BURK
JOANN	GARDNER
DONNA	THOMPSON
DON	BONE
JUAN	FRALEY
DOLORES	WAGNER
MICHEAL	FORMAN
AMBER	DIXON
MELINDA	FERNANDEZ
CONSTAN...	REID
RUBY	WASHING...
GINA	WILLIAMS...

This query uses except. First, it selects all customers who rented action movies. Then, it subtracts all customers who rented either comedy or classics movies.

```

43
44 • select first_name, last_name
45     from customer
46     join rental using (customer_id)
47     join inventory using (inventory_id)
48     join film using (film_id)
49     join film_category using (film_id)
50     join category using (category_id)
51     group by first_name, last_name
52     having sum(category.name = 'Action') > 0
53            and sum(category.name = 'Comedy') = 0
54            and sum(category.name = 'Classics') = 0;
55
56 #4 Which actor has appeared in the most English-l

```

100% 21:53 1 error found

Result Grid Filter Rows: Search Export:

first_name	last_name
LAWRENCE	LAWTON
MATTHEW	MAHAN
TOM	MILNER
JO	FOWLER
SCOTT	SHELLEY
EDWIN	BURK
JOANN	GARDNER
DONNA	THOMPSON
DON	BONE
JUAN	FRALEY
DOLORES	WAGNER
MICHEAL	FORMAN
AMBER	DIXON
MELINDA	FERNANDEZ
CONSTAN...	REID
RUBY	WASHING...
GINA	WILLIAMS...

Through the use of group by on the names of the actors, sum() can be used on the category names.

#4

```
57 with actor_total as (  
58     select first_name, last_name, sum(language.name = 'English') as sum_eng  
59     from actor  
60     join film_actor using (actor_id)  
61     join film using (film_id)  
62     join language using (language_id)  
63     group by first_name, last_name  
64 )  
65 select *  
66 from actor_total  
67 where sum_eng = (select max(sum_eng) from actor_total);  
68
```

0% 9:65 1 error found

Result Grid Filter Rows: Search Export:

first_name	last_name	sum_eng
SUSAN	DAVIS	54

Uses a CTE to get the number of english movies per actor. Then uses max() in a where statement to get the actor with the most.

#5

```
69 #5 How many distinct movies were rented for exactly 10 days from the store where Mike works?  
70 select count(distinct(title))  
71 from film  
72 join inventory using (film_id)  
73 join rental using (inventory_id)  
74 join staff using (staff_id)  
75 where datediff(rental.return_date, rental.rental_date) = 10  
76 and staff.first_name = 'Mike';  
77
```

0% 26:78 1 error found

Result Grid Filter Rows: Search Export:

count(distinct(tit...
47

Uses datediff() in the where statement to subtract rental date from return date.

#6

```

77
78     #6 Alphabetically list actors who appeared in the movie with the largest cast of actors.
79     with largest_film as (
80         select film_id
81         from film_actor
82         group by film_id
83         having count(actor_id) >= all (
84             select count(actor_id)
85             from film_actor
86             group by film_id
87         )
88     )
89     select actor.first_name, actor.last_name, film.title
90     from actor
91     join film_actor using (actor_id)
92     join largest_film using (film_id)
93     join film using (film_id)
94     order by actor.first_name, actor.last_name;
95
00% 40:94 1 error found

```

Result Grid

first_name	last_name	title
BURT	POSEY	LAMBS CINCINATTI
CAMERON	ZELLWEGER	LAMBS CINCINATTI
CHRISTIAN	NEESON	LAMBS CINCINATTI
FAY	WINSLET	LAMBS CINCINATTI
JAYNE	NOLTE	LAMBS CINCINATTI
JULIA	BARRYMORE	LAMBS CINCINATTI
JULIA	ZELLWEGER	LAMBS CINCINATTI
LUCILLE	DEE	LAMBS CINCINATTI
MENA	HOPPER	LAMBS CINCINATTI
MENA	TEMPLE	LAMBS CINCINATTI
REESE	KILMER	LAMBS CINCINATTI
SCARLETT	DAMON	LAMBS CINCINATTI
VAL	BOLGER	LAMBS CINCINATTI
WALTER	TORN	LAMBS CINCINATTI
WOODY	HOFFMAN	LAMBS CINCINATTI

Uses a CTE to get the film with the most actors. Joins actor, film_actor, largest_film, film to get the names of the actors from the largest film. I decided to join film to get the title because if there are films tied for first, this is important.

Primary keys, foreign keys, constraints

```

actor, CREATE TABLE `actor` (
  `actor_id` int NOT NULL,
  `first_name` text,
  `last_name` text,
  PRIMARY KEY (`actor_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

```

```

address, CREATE TABLE `address` (
  `address_id` int NOT NULL,
  `address` text,
  `address2` text,
  `district` text,

```

```
`city_id` int DEFAULT NULL,  
`postal_code` text,  
`phone` text,  
PRIMARY KEY (`address_id`),  
KEY `address_fk` (`city_id`),  
CONSTRAINT `address_fk` FOREIGN KEY (`city_id`) REFERENCES `city` (`city_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

```
category, CREATE TABLE `category` (  
  `category_id` int NOT NULL,  
  `name` text,  
  PRIMARY KEY (`category_id`),  
  CONSTRAINT `category_name` CHECK ((`name` in  
(_utf8mb4'Animation',_utf8mb4'Comedy',_utf8mb4'Family',_utf8mb4'Foreign',_utf8mb4'Sci-Fi',_  
_utf8mb4'Travel',_utf8mb4'Children',_utf8mb4'Drama',_utf8mb4'Horror',_utf8mb4'Action',_utf8mb  
4'Classics',_utf8mb4'Games',_utf8mb4'New',_utf8mb4'Documentary',_utf8mb4'Sports',_utf8mb4  
'Music'))))  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

```
city, CREATE TABLE `city` (  
  `city_id` int NOT NULL,  
  `city` text,  
  `country_id` int DEFAULT NULL,  
  PRIMARY KEY (`city_id`),  
  KEY `city_fk` (`country_id`),  
  CONSTRAINT `city_fk` FOREIGN KEY (`country_id`) REFERENCES `country` (`country_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

```
country, CREATE TABLE `country` (  
  `country_id` int NOT NULL,  
  `country` text,  
  PRIMARY KEY (`country_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

```
customer, CREATE TABLE `customer` (  
  `customer_id` int NOT NULL,  
  `store_id` int DEFAULT NULL,  
  `first_name` text,  
  `last_name` text,  
  `email` text,  
  `address_id` int DEFAULT NULL,  
  `active` int DEFAULT NULL,  
  PRIMARY KEY (`customer_id`),  
  KEY `customer_fk1` (`store_id`),
```

```

KEY `customer_fk2` (`address_id`),
CONSTRAINT `customer_fk1` FOREIGN KEY (`store_id`) REFERENCES `store` (`store_id`),
CONSTRAINT `customer_fk2` FOREIGN KEY (`address_id`) REFERENCES `address`
(`address_id`),
CONSTRAINT `customer_status` CHECK ((`active` in (0,1)))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

```

```

film, CREATE TABLE `film` (
  `film_id` int NOT NULL,
  `title` text,
  `description` text,
  `release_year` int DEFAULT NULL,
  `language_id` int DEFAULT NULL,
  `rental_duration` int DEFAULT NULL,
  `rental_rate` double DEFAULT NULL,
  `length` int DEFAULT NULL,
  `replacement_cost` double DEFAULT NULL,
  `rating` text,
  `special_features` text,
  PRIMARY KEY (`film_id`),
  KEY `film_fk` (`language_id`),
  CONSTRAINT `film_fk` FOREIGN KEY (`language_id`) REFERENCES `language`
(`language_id`),
  CONSTRAINT `film_length` CHECK (((`length` >= 30) and (`rental_rate` <= 200))),
  CONSTRAINT `film_rating` CHECK ((`rating` in
(_utf8mb4'PG',_utf8mb4'G',_utf8mb4'NC-17',_utf8mb4'PG-13',_utf8mb4'R'))),
  CONSTRAINT `film_rent_dur` CHECK (((`rental_duration` >= 2) and (`rental_duration` <= 8))),
  CONSTRAINT `film_rent_rate` CHECK (((`rental_rate` >= 0.99) and (`rental_rate` <= 6.99))),
  CONSTRAINT `film_repl_cost` CHECK (((`replacement_cost` >= 5) and (`replacement_cost`
<= 100))),
  CONSTRAINT `film_spec_feat` CHECK ((`special_features` in (_utf8mb4'Behind the
Scenes',_utf8mb4'Commentaries',_utf8mb4'Deleted Scenes',_utf8mb4'Trailers'))),
  CONSTRAINT `film_year` CHECK (((`release_year` >= 1900) and (`rental_rate` <= 2025)))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

```

```

film_actor, CREATE TABLE `film_actor` (
  `actor_id` int NOT NULL,
  `film_id` int NOT NULL,
  PRIMARY KEY (`actor_id`,`film_id`),
  KEY `film_actor_fk2` (`film_id`),
  CONSTRAINT `film_actor_fk1` FOREIGN KEY (`actor_id`) REFERENCES `actor` (`actor_id`),
  CONSTRAINT `film_actor_fk2` FOREIGN KEY (`film_id`) REFERENCES `film` (`film_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

```



```

rental, CREATE TABLE `rental` (
  `rental_id` int NOT NULL,
  `rental_date` datetime DEFAULT NULL,
  `inventory_id` int DEFAULT NULL,
  `customer_id` int DEFAULT NULL,
  `return_date` text,
  `staff_id` int DEFAULT NULL,
  PRIMARY KEY (`rental_id`),
  UNIQUE KEY `rental_u` (`rental_date`,`inventory_id`,`customer_id`),
  KEY `rental_fk1` (`inventory_id`),
  KEY `rental_fk2` (`customer_id`),
  KEY `rental_fk_staff` (`staff_id`),
  CONSTRAINT `rental_fk1` FOREIGN KEY (`inventory_id`) REFERENCES `inventory`
(`inventory_id`),
  CONSTRAINT `rental_fk2` FOREIGN KEY (`customer_id`) REFERENCES `customer`
(`customer_id`),
  CONSTRAINT `rental_fk_staff` FOREIGN KEY (`staff_id`) REFERENCES `staff` (`staff_id`),
  CONSTRAINT `rental_return` CHECK ((`return_date` between _utf8mb4'1990-01-01' and
_utf8mb4'2025-12-25')),
  CONSTRAINT `rental_start` CHECK ((`rental_date` between _latin1'1990-01-01' and
_latin1'2025-12-23'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

```

```

staff, CREATE TABLE `staff` (
  `staff_id` int NOT NULL,
  `first_name` text,
  `last_name` text,
  `address_id` int DEFAULT NULL,
  `email` text,
  `store_id` int DEFAULT NULL,
  `active` int DEFAULT NULL,
  `username` text,
  `password` text,
  PRIMARY KEY (`staff_id`),
  KEY `staff_fk1` (`address_id`),
  KEY `staff_fk2` (`store_id`),
  CONSTRAINT `staff_fk1` FOREIGN KEY (`address_id`) REFERENCES `address`
(`address_id`),
  CONSTRAINT `staff_fk2` FOREIGN KEY (`store_id`) REFERENCES `store` (`store_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

```

```

store, CREATE TABLE `store` (
  `store_id` int NOT NULL,
  `address_id` int DEFAULT NULL,

```

```
PRIMARY KEY (`store_id`),  
KEY `store_fk` (`address_id`),  
CONSTRAINT `store_fk` FOREIGN KEY (`address_id`) REFERENCES `address`  
(`address_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

```
film_category, CREATE TABLE `film_category` (  
  `film_id` int NOT NULL,  
  `category_id` int NOT NULL,  
  PRIMARY KEY (`film_id`,`category_id`),  
  KEY `film_category_fk2` (`category_id`),  
  CONSTRAINT `film_category_fk1` FOREIGN KEY (`film_id`) REFERENCES `film` (`film_id`),  
  CONSTRAINT `film_category_fk2` FOREIGN KEY (`category_id`) REFERENCES `category`  
(`category_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

```
inventory, CREATE TABLE `inventory` (  
  `inventory_id` int NOT NULL,  
  `film_id` int DEFAULT NULL,  
  `store_id` int DEFAULT NULL,  
  PRIMARY KEY (`inventory_id`),  
  KEY `inventory_fk2` (`store_id`),  
  KEY `inventory_fk1` (`film_id`),  
  CONSTRAINT `inventory_fk1` FOREIGN KEY (`film_id`) REFERENCES `film` (`film_id`),  
  CONSTRAINT `inventory_fk2` FOREIGN KEY (`store_id`) REFERENCES `store` (`store_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

```
language, CREATE TABLE `language` (  
  `language_id` int NOT NULL,  
  `name` text,  
  PRIMARY KEY (`language_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

```
payment, CREATE TABLE `payment` (  
  `payment_id` int NOT NULL,  
  `customer_id` int DEFAULT NULL,  
  `staff_id` int DEFAULT NULL,  
  `rental_id` int DEFAULT NULL,  
  `amount` double DEFAULT NULL,  
  `payment_date` text,  
  PRIMARY KEY (`payment_id`),  
  KEY `payment_fk1` (`customer_id`),  
  KEY `payment_fk2` (`staff_id`),  
  KEY `payment_fk3` (`rental_id`),
```

```
CONSTRAINT `payment_fk1` FOREIGN KEY (`customer_id`) REFERENCES `customer`  
(`customer_id`),  
CONSTRAINT `payment_fk2` FOREIGN KEY (`staff_id`) REFERENCES `staff` (`staff_id`),  
CONSTRAINT `payment_fk3` FOREIGN KEY (`rental_id`) REFERENCES `rental` (`rental_id`),  
CONSTRAINT `pay_date` CHECK ((`payment_date` between _utf8mb4'1990-01-01' and  
_utf8mb4'2025-12-25')),  
CONSTRAINT `payment_amount` CHECK ((`amount` >= 0))  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```